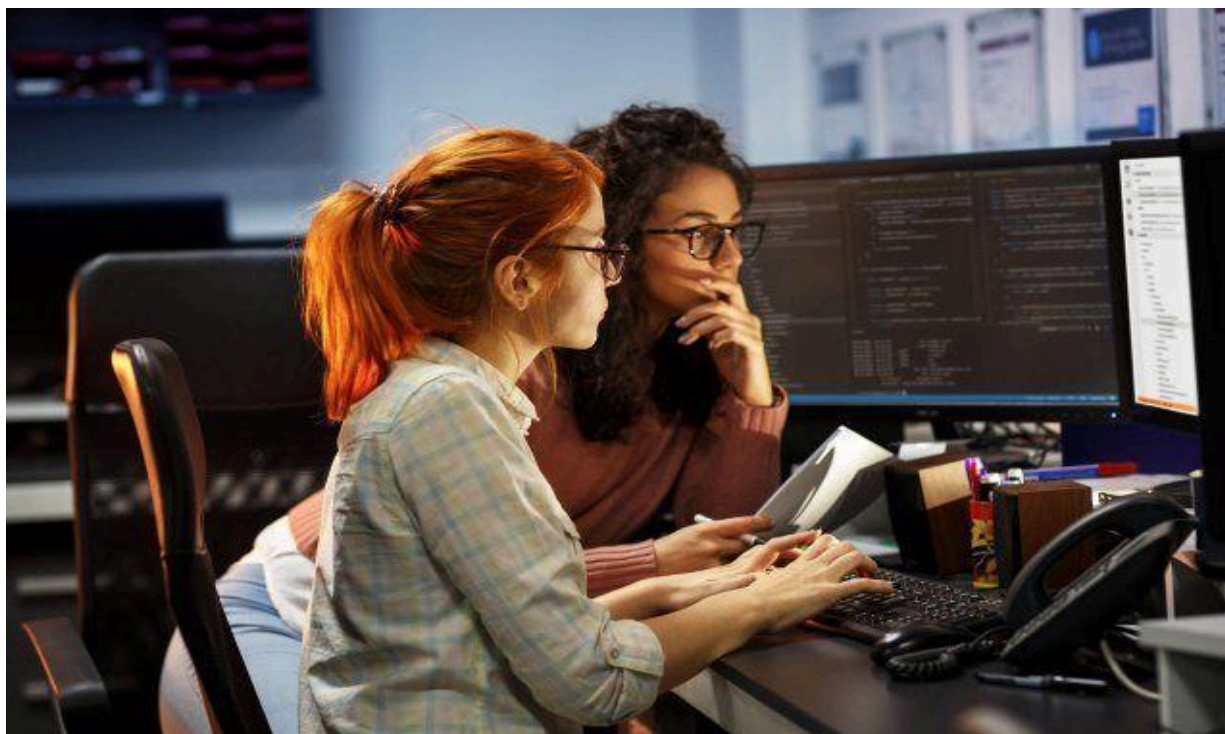




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 3

Tema: Programacion Orientada a Objetos en Desarrollo Moderno.

SEMANA 10

Autor:

Introducción:

Bienvenidos al material de lectura #2 de la Unidad #3 de la materia Programación Orientada a Objetos. En este documento exploramos más acerca de los verbos HTTP, anotaciones como `@RequestBody`, `@PathVariable`, `@RequestParam`, `@Valid` y `@NotNull` y DTOs. Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en los ejemplos, ventajas, limitaciones y ejercicios propuestos, ya que contribuirán significativamente a tu proceso de aprendizaje.

Semana 10

3.7 Métodos HTTP: Las Órdenes que le Das al Mesero

Cuando hablas con nuestro "mesero" (la API REST), no solo le dices qué recurso quieres (ej. "productos"), sino también **qué acción** quieres realizar. Estas acciones son los **Métodos HTTP**. Se corresponden directamente con las operaciones CRUD (Create, Read, Update, Delete) que ya conoces de las bases de datos.

Método HTTP	Operación CRUD	Propósito	Ejemplo de URL
GET	Read (Leer)	 "Tráeme información". Pide datos sin modificar nada. Es seguro repetirlo una y otra vez.	GET <code>/api/products</code> (tráeme todos) GET <code>/api/products/5</code> (tráeme el #5)
POST	Create (Crear)	 "Apunta este nuevo pedido". Envía datos para crear un recurso nuevo. Cada vez que lo envías, creas uno nuevo.	POST <code>/api/products</code> (crea uno nuevo con los datos que te envío)
PUT	Update (Actualizar)	 "Cambia este pedido por completo". Envía datos para reemplazar un recurso existente. Si el recurso no existe, a veces lo crea.	PUT <code>/api/products/5</code> (reemplaza el #5 con estos nuevos datos)
DELETE	Delete (Borrar)	 "Cancela este pedido". Elimina un recurso específico.	DELETE <code>/api/products/5</code> (borra el #5)

GET: Para Consultar Datos

Ejemplo 1: Obtener todos los productos

Se usa para traer la lista completa de recursos.

```
// GET /api/products

@GetMapping

public List<Product> obtenerTodos() {

    // Simplemente le pide al repositorio todos los productos
    y los devuelve.

    return repo.findAll();

}
```

POST: Para Crear Datos Nuevos 

Usa **@RequestBody** para recibir los datos del nuevo producto en formato JSON.

```
// POST /api/products

@PostMapping

@ResponseStatus(HttpStatus.CREATED) // Devuelve un código de
estado 201 (Creado)

public Product crear(@RequestBody Product nuevoProducto) {

    // Guarda el nuevo producto que viene en el cuerpo de la
    petición.

    // El 'id' se genera automáticamente en la base de datos.

    return repo.save(nuevoProducto);

}
```

JSON que enviarías en Postman:

```
{

    "name": "Monitor Curvo",
```

```
"price": 350.00
}
```

PUT: Para Actualizar Datos Existentes

Combina **@PathVariable** para saber *cuál* actualizar y **@RequestBody** para recibir los *nuevos datos*.

```
// PUT /api/products/5

@PutMapping("/{id}")

public Product actualizar(@PathVariable Long id, @RequestBody
Product datosProducto) {

    // 1. Busca el producto existente por su ID.

    return repo.findById(id)

        .map(productoExistente -> {

            // 2. Actualiza los campos del producto existente
            con los nuevos datos.

            productoExistente.setName(datosProducto.getName());

            productoExistente.setPrice(datosProducto.getPrice());

            // 3. Guarda y devuelve el producto actualizado.

            return repo.save(productoExistente);

        }).orElse(null); // Si no lo encuentra, devuelve
null.
}
```

JSON que enviarías en Postman:

```
{
```

```
"name": "Monitor Curvo 27 pulgadas",
"price": 375.50
}
```

DELETE: Para Borrar Datos

Usa **@PathVariable** para identificar el recurso que se va a eliminar.

```
// DELETE /api/products/5

@DeleteMapping("/{id}")

@ResponseStatus(HttpStatus.NO_CONTENT) // Devuelve un código
204 (Sin Contenido)

public void borrar(@PathVariable Long id) {

    // Le pide al repositorio que borre el producto con el ID
    especificado.

    // No devuelve nada en el cuerpo de la respuesta.

    repo.deleteById(id);

}
```

En resumen: Piensa en los métodos HTTP como los verbos de tus peticiones. Le dicen al servidor la **intención** de tu llamada.

3.8 Las Anotaciones: De Dónde Saca la Información el Mesero

Cuando haces una petición, envías datos. Spring necesita saber de dónde tomar esos datos para pasárselos a tus métodos Java. Para eso usamos anotaciones específicas.

@PathVariable

"Sácalo de la propia URL"

Se usa para capturar valores que son parte de la ruta de la URL, como un identificador.

- **Ejemplo de URL:** `GET /api/products/123`
- **Código:**
Java

```
@GetMapping("/{id}")

public Product getProductById(@PathVariable Long id) {

    // Aquí, la variable 'id' tendrá el valor 123.

    return repo.findById(id).orElse(null);

}
```

@RequestParam

"Sácalo de los parámetros de consulta (después del ?)"

Se usa para filtrar, ordenar o paginar. Son los "extras" opcionales de una petición.

- **Ejemplo de URL:** `GET /api/products/search?q=Teclado&stock=true`
- **Código:**
Java

```
@GetMapping("/search")

public List<Product> searchProducts(

    @RequestParam String q,

    @RequestParam(required = false) Boolean stock) {

    // 'q' tendrá "Teclado" y 'stock' tendrá true.

    // (required = false) lo hace opcional.
```

```
}
```

@RequestBody

"Sácalo del cuerpo (body) de la petición"

Se usa cuando envías datos complejos, como un objeto completo en formato JSON para crear o actualizar un recurso. Es exclusivo de métodos como POST y PUT.

- **Ejemplo de Petición:** `POST /api/products` con un cuerpo JSON.
- **Código:**
Java

@PostMapping

```
public Product createProduct(@RequestBody Product newProduct)
{
    // Spring convierte automáticamente el JSON del body
    // en un objeto Java 'newProduct'.
    return repo.save(newProduct);
}
```

3.9 DTOs: La "Comanda" Simplificada del Mesero

Imagina que tu clase `Product` tiene 20 atributos, incluyendo datos internos como `fechaUltimaModificacion` o `usuarioModificador`. Además, un `Product` puede tener una lista de `Reviews` y cada `Review` tiene un `User`, que a su vez tiene más datos.

Problema: Si devuelves el objeto `Product` directamente, expones datos innecesarios, puedes crear bucles infinitos (el producto tiene reviews, la review tiene un producto, etc.) y obligas al cliente a recibir un montón de información que no necesita.

Solución: Usar un **DTO (Data Transfer Object)**.

Un DTO es una clase simple, sin lógica ni conexión a la base de datos, diseñada exclusivamente para **transferir datos** entre el cliente y el servidor. Es como una "comanda" simplificada que el mesero usa para comunicarse con la cocina y contigo.

Ejemplo:

Tu Entidad (el objeto real de la cocina):

Java

@Entity

```
public class Product {  
    @Id private Long id;  
    private String name;  
    private double price;  
    private LocalDateTime internalAuditTimestamp; // No  
    queremos exponer esto  
    // ... getters/setters  
}
```

Tu DTO (la "comanda" que ve el cliente):

Java

```
public class ProductDTO {  
    private Long id;  
    private String name;  
    private double price;  
    // Solo los datos que el cliente necesita.  
    // ... getters/setters  
}
```



```
}
```

En el Controlador: Conviertes la entidad a DTO antes de enviarla.

Java

```
@GetMapping("/{id}")

public ProductDTO getProductById(@PathVariable Long id) {

    Product product = repo.findById(id).get(); // Obtienes el
    objeto real

    // Lo conviertes a DTO para la respuesta

    ProductDTO dto = new ProductDTO();

    dto.setId(product.getId());

    dto.setName(product.getName());

    dto.setPrice(product.getPrice());

    return dto; // Envías la versión simplificada y segura

}
```

Beneficios Clave: Seguridad, eficiencia y flexibilidad. Puedes tener diferentes DTOs para diferentes situaciones (ej. `ProductSummaryDTO`, `ProductDetailDTO`).

3.10 Validación: El Mesero Revisa tu Pedido

¿Qué pasa si un cliente intenta crear un producto sin nombre o con un precio negativo? Tu aplicación podría fallar o guardar datos basura. La validación consiste en establecer reglas para los datos de entrada.

Spring Boot lo hace muy fácil con las anotaciones de `jakarta.validation`.

1. **Añade las Reglas a tu DTO de Entrada:** Usamos un DTO también para la entrada de datos (`CreateProductDTO`).

Java

```
import jakarta.validation.constraints.*;

public class CreateProductDTO {

    @NotNull(message = "El nombre no puede ser nulo")

    @Size(min = 3, max = 50, message = "El nombre debe tener
entre 3 y 50 caracteres")

    private String name;

    @NotNull(message = "El precio no puede ser nulo")

    @Positive(message = "El precio debe ser un número
positivo")

    private Double price;

    // Getters y Setters...
}
```

2. Activa la Validación en el Controlador con **@Valid**: Java

```
@PostMapping

@ResponseStatus(HttpStatus.CREATED)

public ProductDTO createProduct(@Valid @RequestBody
CreateProductDTO dto) {

    // La anotación @Valid le dice a Spring:

    // "Antes de ejecutar este método, revisa si el 'dto'

    // cumple con todas las reglas de validación".

    // Si la validación falla, Spring automáticamente detiene

    // la ejecución y devuelve un error 400 Bad Request
```

```
// con los detalles del fallo.  
  
// Si la validación es exitosa, el código continúa...  
  
Product newProduct = new Product();  
newProduct.setName(dto.getName());  
newProduct.setPrice(dto.getPrice());  
repo.save(newProduct);  
  
// ... convertir a ProductDTO y devolver  
  
}
```

Al usar `@Valid`, tu lógica de negocio solo se ejecuta con datos que ya sabes que son correctos, haciendo tu código mucho más limpio y robusto.

